DOCTrace: Technical Document

Nov 14, 2026

Summary

DOCTrace turns a recorded click-through of any software into a step-by-step guide. Sensitive data is blacked out on the device, and an AI model describes each step. It started as a Chrome extension using cloud AI (Gemini, then Groq). It ended as **DOCTrace Desktop 3.0.0**, a Windows app that runs fully offline with Ollama and whisper.cpp.

- **Final product:** DOCTrace Desktop 3.0.0 (Electron 33, Windows 64-bit), released on GitHub on 11 Nov 2026 as `DOCTrace-Desktop-3.0.0-Windows.zip`.
- **Offline by design:** the app only talks to localhost ports 11434 (Ollama), 1234 (LM Studio) and 8080 (llama.cpp). Cloud providers and API keys were removed in 3.0.0.
- **AI model in use:** `qwen2.5vl:3b` in Ollama, on the CPU. One step takes about 3–5 minutes on the test PC (CPU only).
- **Quality fix that mattered most:** on the same 5-step LMS recording, version 2.0.1 got 0 of 5 steps from the AI (all fell back to templates). After the 2.0.2 fixes, 4 of 5 steps were AI-written with every field filled.
- **Checks:** `tools/check.js` runs 229 checks (syntax, wiring, offline rules, unit tests). All passed on 14 Nov 2026.

Goal and scope

A user performs a task once (for example, submitting an assignment on a university LMS) and explains it out loud. DOCTrace records it and produces a ready-to-share tutorial.

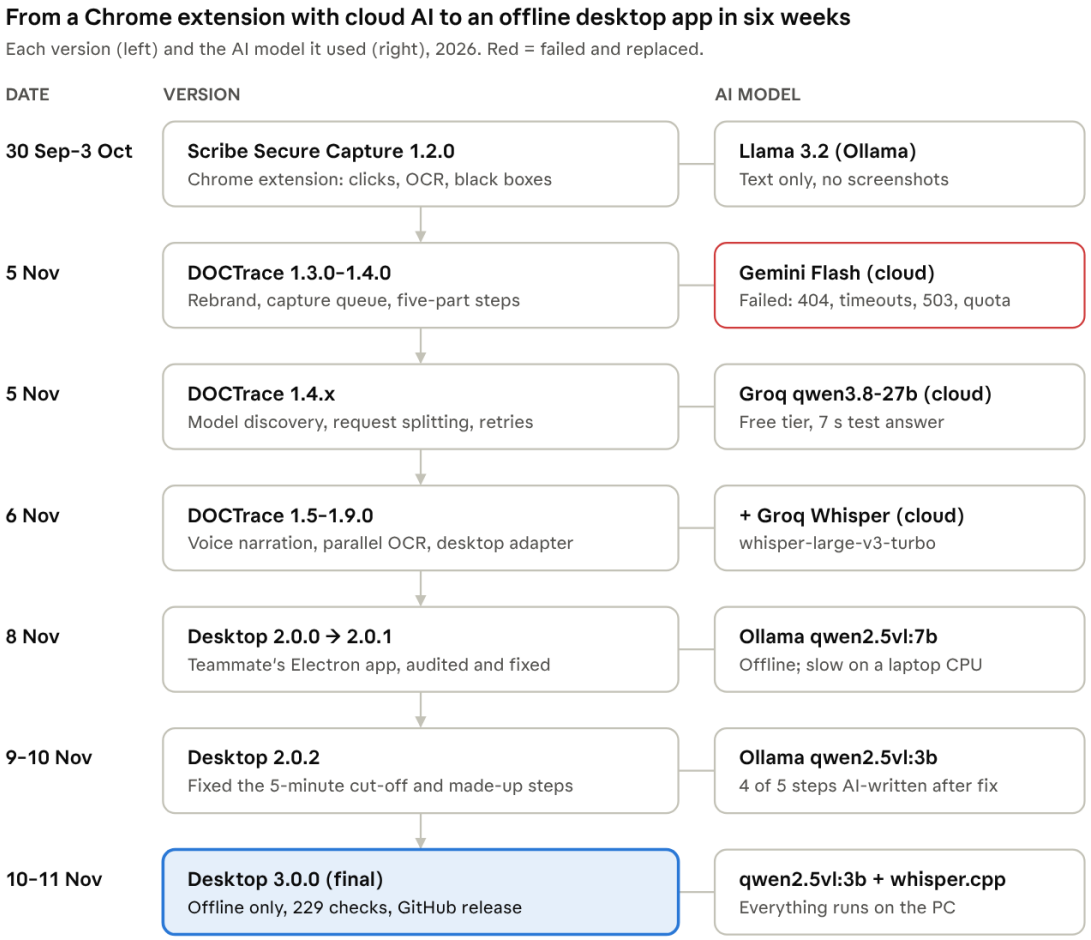
What one recording produces

Output	Content
Steps	One per click; an "observe" snapshot after 7 s without input; a final "complete" step at Stop
Screenshot per step	Redacted copy only, with a red circle on the click
Description per step	Title, instruction, where to find it, screen context, expected outcome, note
Narration	What the user said, matched to each step, plus a spoken intro
Files	`manual.md`, `manual.html`, `manual.pdf`, `manual_manifest.json`, `step_N.png`, `narration.wav`

Privacy promise

- Raw screenshots are never saved or sent. Each frame is redacted on the device first; if redaction fails, the frame is dropped (fail-closed).
- Typed values and keystrokes are never stored.
- In the final version nothing leaves the PC: the AI and speech-to-text run locally.
- One known exception: `narration.wav` is the user's real voice and is not scrubbed. Users are told not to say passwords aloud.

Project timeline

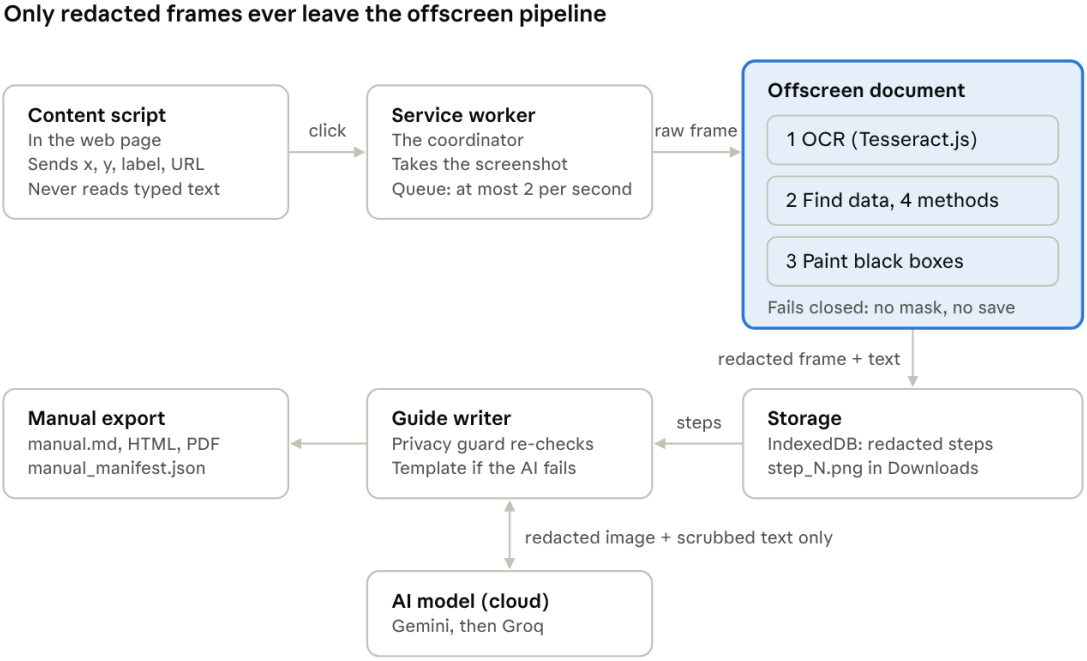


project timeline · 7 milestones, 30 Sep – 11 Nov 2026

The AI model changed as often as the product did: Gemini failed on its free tier and was replaced by Groq on the same day, and the move to the desktop is what made a fully local model possible.

Phase 1: Chrome extension architecture

The first product was a Manifest V3 Chrome extension, built by two groups to one agreed function. Group 1 recorded clicks and screenshots; Group 2 built `processFrame(frame)`, which turns a raw screenshot into a redacted one. Raw screenshots stay in memory and are never stored.



Chrome extension (Manifest V3) · data flow for one click

One click: the content script sends the click to the service worker, which takes a screenshot and hands it to the offscreen document. Only the redacted result is stored, written up by the guide writer and, in the extension, sent to a cloud model. The popup, Sessions, Viewer and Options pages start recordings, show and edit guides, and hold settings.

Chrome limits that shaped the design

- A Manifest V3 service worker has no DOM and cannot start Web Workers, so OCR runs in an offscreen document. Chrome allows only one per extension, and it is also used for the microphone.
- `captureVisibleTab` runs only in the service worker and allows about 2 calls per second, so captures are queued.
- Chrome can stop an idle service worker at any time, so recording state is kept in `chrome.storage`, not in memory.
- An extension sees only browser tabs. It can tell that Chrome lost focus, but not which program took it. This is the main reason the project moved to a desktop app.

Group 1 requirements at the end of the extension phase

Requirement	Status	Note
Click coordinates	Done	`pointerdown`, with DPR and viewport width
Session clock (`tMs`)	Mismatch	Group 1 used ms since start; the merged recorder used wall-clock time

Requirement	Status	Note
Scroll pauses	Designed	500 ms debounce, not built
Application focus	Partly possible	Tab and window focus only, not other programs
Video timeline	Not started	Would need `tabCapture` + MediaRecorder in the offscreen page
Local event buffering	Not started	Planned outbox in IndexedDB with retries

Privacy pipeline (on the device)

Every screenshot passes through one function, `processFrame(frame)`, before anything is saved: raw frame in, redacted frame plus scrubbed text out. It was built by Group 2 to the shared contract and is unchanged in spirit from the extension to 3.0.0 (now under `core/engine/`).

1. **OCR:** Tesseract.js 5.1.1 (WASM, bundled locally, English language file) reads the text and its boxes. It is given a fixed 300 DPI, which stopped the "Estimating resolution" warnings.
2. **Detection, four methods:** regular expressions, keyword labels, spatial rules (a value beside or below a label such as "Password:") and a small ML classifier. Types include email, phone, CNIC, passport, SSN, password, API key, card number, IBAN and bank account.
3. **Masking:** solid black boxes are painted on a copy of the frame (OffscreenCanvas). The red click circle is drawn on the redacted copy.
4. **Fail-closed:** if any stage fails, the frame is not saved.
5. **Text scrubbing:** screen text and narration are scrubbed into tokens such as `[REDACTED:email]`.
6. **Privacy guard before the AI:** every value sent to a model is scrubbed again, then checked by a third pass. If anything sensitive or any raw image data is found, the request is refused. The AI's answer is scrubbed too, and tokens are turned into readable words ("your email address").

Known gap: detection only finds text OCR can read. Text inside photos, or non-English text, can be missed, so users are told to check steps before sharing.

Phase 2: AI step descriptions with cloud models

Without an AI, steps get generic template text ("Type the required information in the ... field"). Making the AI step writer reliable took most of the project. Gemini's free tier failed in four different ways in one day, so Groq became the default on 5 Nov 2026.

First trial (3 Oct): Group 2 connected the extension to Llama 3.2 through a Python Flask backend and Ollama. Llama 3.2 1B/3B is text-only, so it saw element labels and page text, not the screenshot.

DOCTrace 1.3.0 and 1.4.0 (5 Nov): the extension was renamed from "Scribe Secure Capture" to DOCTrace and connected to Google Gemini, which reads the redacted screenshots. Two other changes landed at the same time:

- Fast clicks no longer drop screenshots. Chrome allows about 2 `captureVisibleTab` calls per second; one shared queue now spaces captures and retries on quota errors.
- Each step got five parts (title, instruction, where to find it, screen context, expected outcome).

What failed and what we changed

Provider	Problem seen	Cause	Fix
Gemini	HTTP 404 NOT_FOUND	`gemini-2.5-flash` no longer available to the key	Back to the alias `gemini-flash-latest`; retry once on 404
Gemini	"Did not answer within 28 s"	The newest Flash model "thinks"; one answer took 30–60 s	Longer timeout; requests run from the offscreen page, so Chrome's 30 s service-worker limit no longer kills them
Gemini	HTTP 503 overloaded	Google's side, at peak times	Longer back-off, then fall back to `gemini-flash-lite-latest`
Gemini	"Daily quota exhausted"	Free tier about 20 requests per model per day	Switched provider
Groq	HTTP 404 model_not_found	Llama 4 Scout was retired	Ask Groq for its current model list; default `qwen/qwen3.8-27b` (up to 3 images per request)
Groq	HTTP 413 rate_limit_exceeded (tokens)	Each screenshot counts as about 2,000 tokens; 3 images went over the free tier's per-minute limit	Split oversized requests down to one image and remember the size that worked
Groq	HTTP 429 rate limit	Free-tier requests per minute	Wait for `Retry-After`, up to 5 times

Result: Groq's test connection answered in 7 s with a correct description ("Click 'Settings' in the left-hand navigation menu to access application preferences."). Mistral was kept as a backup provider. Error messages show the real reason but never the API key.

Phase 3: Voice narration

Voice was added on 6 Nov 2026 so that what the user says ("only ZIP and PDF files are accepted") ends up in the guide. It is transcribed with Whisper, matched to steps by time, scrubbed, and used by the AI as the main source for each step.

How it works

1. **Recording:** the microphone records during the whole session. The extension recorded compressed Opus audio in its offscreen page; the desktop app records 16 kHz mono WAV (`narration.wav`, about 1.83 MB per minute).

2. **Transcription:** Whisper with timestamps. Extension: `whisper-large-v3-turbo` on Groq. Desktop: Groq or local whisper.cpp; 3.0.0: whisper.cpp only (`whisper-cli.exe` with a `ggml-base` model, English).
3. **Matching to steps:** each sentence is attached to the click it describes. Speech before the first click becomes the guide's spoken intro.
4. **Scrubbing:** every sentence passes the same privacy filter as screen text before it is stored or sent to the AI.
5. **Writing:** the AI follows the narration; extra details (requirements, limits, tips) go into a new **Note** field.

Problems found in test recordings and fixes

- Recordings 6 and 7 had no usable transcript: Whisper was given silence. Fix: a microphone picker with a live level bar, a "mic silent!" warning while recording, and silence is never sent for transcription.
- Whisper now starts the moment Stop is pressed, while the last screenshots are still being processed (1.9.0).
- A strict "time window" matching rule was tested against Recording 9 and put three sentences on the wrong step, so the smarter rule stayed the default.

Matching v4 (built by a teammate in the desktop app, ported back into extension 1.10.0 on 8 Nov):

- Word-level timestamps; a pause of 0.8 s or more starts a new phrase.
- Speech in the 2.5 s before a click announces that click; speech during a click belongs to it.
- Instructions point forward ("Click Submit." goes to the next step).
- The microphone's input delay is subtracted (Bluetooth adds 150–300 ms), and slow audio-clock drift is corrected on long recordings.

Phase 4: Offline Windows desktop app (2.0.0 and 2.0.1)

A browser extension can only see clicks inside web pages, so the project moved to a desktop app that records any program. A teammate built DOCTrace Desktop 2.0.0 in Electron on top of the extension's 1.9.0 engine; we audited it on 8 Nov 2026 and shipped a fixed 2.0.1 the same night.

Preparation in the extension (1.9.0): all Chrome calls in the engine (storage, capture, file saving, AI requests) were moved behind one `platform` adapter, with a porting guide for Electron and Tauri.

What the desktop app changed

Area	Extension	Desktop 2.0.0
Clicks	Content script inside web pages	System-wide mouse/keyboard hook (`uiohook-napi`), any program
Screenshots	Visible browser tab	Whole screen, cropped to the focused window (window found by a hidden PowerShell script)
Voice	Opus, Whisper on Groq	16 kHz WAV, Groq or local whisper.cpp

Area	Extension	Desktop 2.0.0
Step writer	Gemini, Groq, Mistral; local models text-only	Same, plus Ollama reading the screenshots locally
Editing	Edit text, rewrite	Also reorder and delete steps
Files	Downloads folder	`Documents\DOCTrace\Recordings`

Audit findings (8 Nov). The engine was identical to ours except three voice files, and it passed all 56 extension tests. Problems found, all fixed in 2.0.1:

Severity	Problem	2.0.1 fix
High	The Groq key was erased on every Save when Groq was selected (reproduced)	Save keeps the key typed under Voice narration
High	The microphone recorded by default, without consent	Records only after "Record my voice" is ticked and saved
Medium	Local models got 120 s per step and two requests at once, so CPU steps timed out	600 s for localhost, one request at a time
Medium	Groq transcription failed after about 13.7 min (25 MB upload limit)	WAV split into valid 10-minute pieces, results merged
Medium	Main process would run any program as "whisper" and reach any localhost port	Only `whisper-cli(.exe)` with a `.bin` model; only ports 11434, 1234, 8080
Medium	"Starting..." could hang forever	Each start-up step has a time limit and names itself in the error
Medium	Antivirus ransomware protection silently held writes to Documents	Falls back to `%LOCALAPPDATA%\DOCTrace\Recordings` within 4 s
Low	Electron fuses allowed running the exe as plain Node or with a debugger	RunAsNode, NODE_OPTIONS and inspect fuses switched off

The source code was not available, so 2.0.1 was made by unpacking `app.asar`, patching the built JavaScript, repacking it with fresh SHA-256 integrity data, and flipping the fuse bytes in the exe.

Deployment problems on the test PC (9 Nov): Avast and Reason Cybersecurity deleted or blocked `DOCTrace Desktop.exe` because a global input hook plus hidden PowerShell looks like a keylogger. Exceptions did not help; both were uninstalled and Windows Defender left the app alone. The Documents folder was OneDrive-synced, so recordings were pointed at `%LOCALAPPDATA%` to keep them off the cloud.

Phase 5: Fully offline AI with Ollama (7b, then 3b, then 2.0.2)

Groq is a cloud service, so the offline version needed a vision model that runs on the PC. We chose Qwen2.5-VL in Ollama, first the 7B model and then the 3B one. The 3B model only gave useful guides after the 2.0.2 fixes to the connection and the prompt.

Why Qwen2.5-VL. It is from the same Qwen family as the Groq model, and it reads on-screen text (buttons, menus, forms) well. The larger Qwen 3.8 exists in Ollama only as 27B, too heavy for 16 GB of RAM without a graphics card. Ollama needs `OLLAMA_CONTEXT_LENGTH=8192`: the default 4,096 tokens is too small for the instructions plus one screenshot, and the OpenAI-style endpoint cannot set it per request.

7B on a laptop (8 Nov). On an i7-8565U laptop (4 cores, 15.7 GB RAM, no usable GPU) a 21-step guide ran for a long time with no progress shown. Steps that ran into the old 120 s limit fell back to templates. We added an "x of y steps done" progress line and recommended the 3B model, about twice as fast (an estimate, not measured).

3B on the test PC (9–10 Nov). Xeon E5-1620 v2 (4 cores / 8 threads, no AVX2), 16 GB RAM; the Quadro 2000 is too old for Ollama, so everything runs on the CPU. Measured:

- Text-only test, including loading the 3 GB model: 18 s.
- One synthetic 1024 px screenshot with a short prompt: 161 s (1,137 prompt tokens, 58 answer tokens).
- Ollama read prompts at about 15 tokens/s and wrote answers at about 5–6 tokens/s.

First real guide failed. Recorded on the Claude chat window, step 1 described words from a chat message elsewhere on the screen ("fake LMS", "Upload file", a sleep-mode note) instead of the clicked text. Steps 2–4 got template text. A second guide on the Bahria LMS got template text for all 5 steps.

Root causes, found in Ollama's log and by replaying step 2 with the exact request:

1. **Requests cut off at exactly 5:00.** Node's `fetch` waits at most 300 s for a reply, whatever DOCTrace's 600 s limit says. Ollama logged `500 | 5m0s` and cancelled the task.
2. **The cloud prompt was too complex for 3B.** Given about 3,000 tokens of rules and JSON, the model echoed the input back (632 tokens, 7 min), left out the instruction field, copied the example ("semester courses and class schedules") and the previous step, and numbered step 2 as step 1. DOCTrace discarded the answer.
3. **Too much screen text.** 1,500 characters were sent; the misleading words sat at character 1,325, the clicked words at 519.

2.0.2 fixes (10 Nov)

- Requests to localhost go through `node:http`, which has no 5-minute limit.
- A short prompt only for local vision models: one step, the facts as plain text (click words first, screen text marked as background), no previous steps, a six-field answer with placeholders and no step number, at most 450 answer tokens.
- Answers with a wrong step number are matched by order; a bare JSON object is accepted.
- Local models get 500 characters of screen text; a note is kept only when the user actually spoke.

- The prompt says to describe only what is under the red circle, and that chats or documents on screen are content, not buttons.

Before and after

Measure	2.0.1	2.0.2 / 3.0.0
Prompt size for one step	about 3,000 tokens	about 1,600–1,700 tokens
Answer length	450–630 tokens	about 90 tokens
Time per step on the test PC	4–7 min, some cut at 5:00	about 3–5 min
AI-written steps, same 5-step LMS guide	0 of 5	4 of 5 (the final "complete" step stays a template)
Made-up content in the replayed step 1	Yes	None seen

The rewritten LMS guide now reads, for example: "Click on the 'Assignments' tab in the left sidebar to view your assignments", with where to find it and the expected outcome filled in.

Current version: DOCTrace Desktop 3.0.0

3.0.0 (10 Nov 2026) is the single final product: offline only, generic for any program, with its code kept as editable source and rebuilt by a script. It keeps every 2.0.1 and 2.0.2 fix.

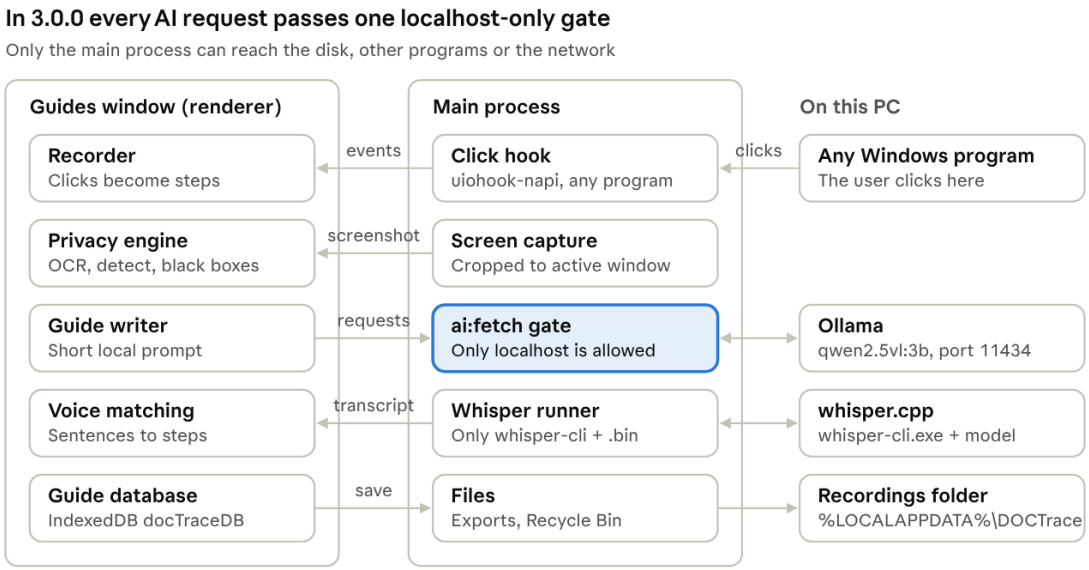
What changed from 2.0.2

- **Cloud removed:** Groq, Mistral, Gemini, Anthropic and cloud transcription are gone, with their API key fields and consent boxes. Keys saved by older versions are deleted on first start.
- **Local writers:** Ollama reading screenshots is the default (recommended). Ollama text-only (`qwen2.5:7b`) and LM Studio now work; before, their requests were blocked.
- **Generic wording:** the AI instructions use examples for any program or website, not only a university LMS.
- **Files:** new guides go to `%LOCALAPPDATA%\DOCTrace\Recordings` (not synced by OneDrive). A PC that already has guides in Documents keeps using it.
- **Deleting:** deleting a guide moves its folder to the Recycle Bin. Screenshots of deleted or moved steps also go to the Recycle Bin, and "Transcribe again" still matches narration after steps were moved.

How offline is enforced

- The main process allows network requests only to `localhost`, `127.0.0.1` or `[::1]` on ports 11434, 1234 and 8080 (`checkUrl`).
- The windows' Content-Security-Policy (`connect-src 'self'`) blocks direct requests, so all AI traffic must go through one IPC call, `ai:fetch`.
- Tesseract's worker, core and English language file are bundled.

Architecture



DOCTrace Desktop 3.0.0 · processes and local services

The guides window runs the engine but has no disk or network access of its own; it asks the main process over IPC. The main process records clicks in any program, captures the screen, saves files and is the only path to Ollama and whisper.cpp.

Code layout

Path	Role
`DOCTrace Desktop/`	Runnable app: Electron 33.2.1 runtime (fuses off), `resources/app.asar`, native click hook
`app-source/out/main/index.js`	Main process: windows, tray, click hook, screen capture, files, `aiFetch`, whisper.cpp, PDF export, Recycle Bin
`app-source/out/renderer/assets/`	Guides window (recorder, editor, Settings) and the floating Record bar
`app-source/out/renderer/core/engine/`	Engine: OCR, detection, redaction, recorder, guide writer, voice, storage
`tools/`	`check.js` (229 checks), `pack-asar.js`, `extract-asar.js`

Release: GitHub repository `Umer1837/DOCTrace-Desktop`, release v3.0.0 with the asset `DOCTrace-Desktop-3.0.0-Windows.zip` (11 Nov 2026).

Model comparison

Gemini Flash wrote the best descriptions in our evaluation, but its free tier was unreliable. Groq's Qwen model was the working cloud choice. For the offline product, `qwen2.5vl:3b` is the only tested model that reads screenshots, fits a CPU-only PC and gives usable steps (after 2.0.2).

Model	Runs where	Reads screenshots	What we observed	Outcome
`qwen2.5vl:3b` (Ollama)	Local, CPU	Yes	3–5 min per step on the test PC; 4 of 5 steps AI-written after 2.0.2; nothing made up in the replay	**Final choice (3.0.0)**
`qwen2.5vl:7b` (Ollama)	Local, CPU	Yes	Minutes per step on a 4-core laptop; steps timed out under the old 120 s limit	Default name in 3.0.0; too slow on weak PCs
`qwen/qwen3.8-27b` (Groq)	Cloud, free tier	Yes, up to 3 per request	7 s test answer; needed request splitting (HTTP 413)	Default in extension 1.4–1.10
Gemini Flash (`gemini-flash-latest`)	Cloud	Yes	Best on case_02 (below); then 404, timeouts, 503 and a ~20 requests/day quota	Replaced by Groq
Groq (model not recorded)	Cloud	—	11-step manual: 11/11 clear instructions, but described the screen in only 2/11 steps	Evaluation only
Llama 3.2 1B/3B	Local, CPU	No (text only)	11/11 steps had an instruction; repeated the previous action in 5/11; slow on CPU	Not used: cannot see screenshots
Moondream	Not recorded	Yes	0/11 steps said what to click; misspelled the university in 4 steps; added details a screenshot cannot show	Rejected
Nemotron Nano Omni 30B	Cloud, free tier	Yes	case_02: named the page but left out the click; slowest (9.0 s)	Rejected
Grok	Cloud, paid	—	Not tested (paid API)	—
LLaVA, Gemma	—	—	Not scored	—

Caption scores on test case case_02 (the login page's Password field, one reference description):

Model	ROUGE-L	METEOR	BLEU-4	Seconds	Description
Gemini Flash	0.436	0.334	0.000	3.5	Correct page and field, 26 words
Nemotron	0.274	0.263	0.000	9.0	Page only, no click
Qwen 3.8 27B	0.248	0.260	0.000	3.3	Correct field, 60 words vs 13 in the reference

These scores are preliminary: one test case, no SPICE or BERTScore, and LLaVA and Gemma were not scored. The local Qwen2.5-VL models were not scored on case_02.

Testing and verification

Every change was covered by automated tests, and the offline AI was checked by replaying real recorded steps against Ollama. Real-use testing is thinner: two guides on the test PC, both without voice.

Stage	Automated checks	What they cover
Group 2 engine (Oct)	14 headless-Chromium tests (reported by Group 2, not re-run by us)	OCR, detection, masking; 28 network requests, 0 external
Extension 1.3.0 → 1.10.x (5–8 Nov)	Node tests grew from 19 to 68	Capture queue, Gemini/Groq clients (404, 413, 429, timeouts), voice matching v3/v4, parallel OCR, platform adapter
Desktop 2.0.1 (8–9 Nov)	16 patch tests	Groq key, consent, localhost ports, whisper allowlist, 25-min WAV split (pieces at 0/600/1200 s), Documents fallback with a write that never returns
Desktop 2.0.2 (10 Nov)	30 patch tests	Plus local HTTP without the 5-minute limit, answer matching, the short local prompt
Desktop 3.0.0 (now)	`tools/check.js`: 229 checks	Syntax, links, IPC wiring, offline and wording rules, unit tests incl. the main process; all pass on 14 Nov 2026

Replays against Ollama: step 2 of a real guide was resent with the 2.0.1 prompt to see the raw answer (the echoed JSON), and step 1 with the 2.0.2 prompt (no made-up content). Ollama's own log was used to read prompt sizes, answer sizes, times and the 5:00 cut-offs.

Test recordings

- 2 Oct (extension): Recordings 1–5 gave 0, 6, 6, 7 and 27 screenshots; repeated clicks in one field made duplicate steps.
- 5–6 Nov (extension): Recordings 5–9 drove the Gemini, Groq and voice fixes.
- 9–10 Nov (desktop, test PC): a 4-step guide on the Claude window and a 5-step guide on the Bahria LMS (Chrome).

Not verified yet

- Voice narration with whisper.cpp in a real guide on the test PC (both guides there had voice off).
- Recordings longer than about 20 minutes.
- Other PCs, a graphics card, and the 7B model end to end with 2.0.2 or later.

Known limitations and next steps

The biggest limit is speed: on a CPU-only PC a 10-step guide takes roughly 30–50 minutes to write. The next most important gaps are code signing and a scored quality test of the local model.

Limitations

Area	Limitation
Speed	3–5 min per step on the CPU; sleep or shutdown stops the writing, and "Write guide with AI" rewrites every step
Quality	The 3B model writes generic text when it lacks click hints; the final "complete" step stays a template
Language	English only (whisper runs with `-l en`, OCR uses the English data)
Privacy	OCR cannot read text inside photos; `narration.wav` is not scrubbed
Platform	Electron 33 is out of support; the exe is not code-signed, so SmartScreen warns on first start
Antivirus	The global input hook and a hidden PowerShell call (to read the active window) look like a keylogger to some antivirus products

Next steps

- ☐ Score `qwen2.5vl:3b` on case_02 and the 11-step manual with the same metrics as the cloud models.
- ☐ Write the final "complete" step with the AI too.
- ☐ Keep the PC awake while a guide is written, and rewrite only the steps still missing.
- ☐ Replace the PowerShell window lookup with a native module, then sign the exe and move to a supported Electron version.
- ☐ Test voice narration with whisper.cpp end to end, and a recording longer than 20 minutes.
- ☐ Try a PC with a graphics card, and a second small model (`gemma3:4b`) for comparison.

Appendix: running the offline version

DOCTrace works without any setup (recording, redaction and template text). The steps below add AI-written steps and voice narration; they take about 20–30 minutes once.

1. **Start the app:** unzip `DOCTrace-Desktop-3.0.0-Windows.zip`, open `DOCTrace Desktop`, double-click `DOCTrace Desktop.exe`. On "Windows protected your PC", click More info, then Run anyway. The app runs in the tray.

2. **Install Ollama** from ollama.com, then in Command Prompt:

   ```
   setx OLLAMA_CONTEXT_LENGTH 8192
   ollama pull qwen2.5vl:3b
   ```

   Quit Ollama from the tray and start it again from the Start menu, so it reads the new setting.
3. **Settings → Guide writer:** tick Use AI, choose "Ollama, reads the screenshots", model name `qwen2.5vl:3b`, Save. It should say Ready.
4. **Voice (optional):** download `whisper-bin-x64.zip` from the whisper.cpp releases and a `ggml-base.en.bin` model. In Settings → Voice narration, tick Record my voice, enter the paths to `whisper-cli.exe` and the model (without quote marks), pick the microphone, Save.
5. **Record:** click Record, click through the task while explaining it, then Stop (or Ctrl+Shift+S). Keep the PC on and plugged in while the guide is written.
6. **Edit and export:** edit, move or delete steps, then Export to PDF.

Where files are saved: `%LOCALAPPDATA%\DOCTrace\Recordings\Recording_YYYY-MM-DD_HH-mm\` holds `manual.md`, `manual.html`, `manual.pdf`, `manual_manifest.json`, `step_N.png` and `narration.wav`. The guide database and settings are in `%APPDATA%\DOCTrace Desktop\`.

Rebuilding after a code change: run `node tools/check.js`, quit DOCTrace, then `node tools/pack-asar.js app-source "DOCTrace Desktop/resources/app.asar.unpacked" "DOCTrace Desktop/resources/app.asar"`.